



RNDr. Richard Ostertág, PhD.
KI FMFI UK Bratislava
ostertag@dcs.fmph.uniba.sk

KI FMFI UK Bratislava

ostertag@dcs.fmph.uniba.sk

Sources

- D3.js home page
 - <http://d3js.org/>
- API Reference
 - <https://github.com/mbostock/d3/wiki/API-Reference>
- Interactive Data Visualization for the Web
 - by Scott Murray
 - <http://chimera.labs.oreilly.com/books/12300000000345>
- Data Visualization with D3.js Cookbook
 - by Nick Qi Zhu
 - <http://nickqizhu.github.io/d3-cookbook/>

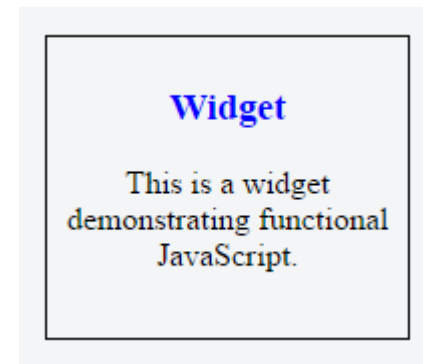
Basic D3 development environment

```
<!doctype html>
<html>
<head>
  <meta charset="utf-8">
  <title>D3 project</title>
  <link rel="stylesheet" href="styles.css">
  <style> <!-- Add CSS content here --> </style>
</head>
<body>
  <!-- Add HTML content here -->
  <script src="https://cdnjs.../d3.js"></script>
  <script>
    window.d3 ||
    document.write('<script src="d3.js"></script>')
  </script>
  <script> <!-- Add JS content here -->
    d3.select("body")
      .append("div")
      .text("Hello world!");
  </script>
</body>
</html>
```

- self-sufficient
- `<meta charset="utf-8">`
 - Uncaught SyntaxError: Unexpected token ILLEGAL
 - $\var \hat{\mu} = 1e-6$
 - $\var \varepsilon = 1e-6$
- <https://cdnjs.com/>
 - uptime > 99.85%
 - do not forget fallback
- `<\script>`
 - otherwise “</” (end-tag open delimiter) would end the script

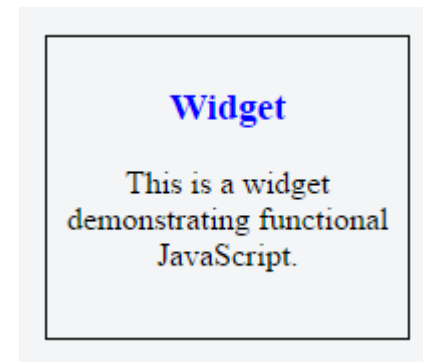
D3-style JavaScript

- ```
function widget(spec) {
 var instance = {};
```
- ```
  var description;  
  instance.description = function(d) {  
    if (!arguments.length) return description;  
    description = d;  
    return instance;  
  };
```
- ```
 instance.render = function() {
 var div = d3.select('body').append("div")
 .attr("class", "box");
 div.append("h3").text(headline)
 .attr("style", "color:" + spec.color);
 div.append("p").text(description);
 return instance;
 };
```
- ```
  return instance;  
}
```
- ```
var w = widget({ color: "brown" })
 .headline("Widget")
 .description("This is ... JavaScript.");
w.render();
```
- ```
<div class="box" style="color:brown">  
  <h3>Simple Widget</h3>  
  <p>This is ... JavaScript.</p>  
</div>
```
- <https://jsfiddle.net/RiOS/b6ned218/>



D3-style JavaScript

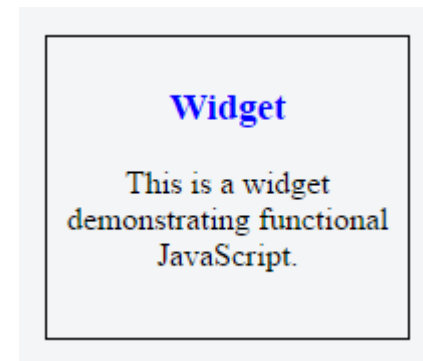
- function widget(spec) {
var **instance** = {};
var description;
instance.description = function(d) {
if (!arguments.length) return description;
description = d;
return instance;
};
instance.render = function() {
var div = d3.select('body').append("div")
.attr("class", "box");
div.append("h3").text(headline)
.attr("style", "color:" + spec.color);
div.append("p").text(description);
return instance;
};
return instance;
}
- New object is created using object literal.
- widget({ color: "brown" })
instance("Widget")
.description("This is ... JavaScript.");
w.render();
• `<div class="box" style="color:brown">
<h3>Simple Widget</h3>
<p>This is ... JavaScript.</p>
</div>`
• <https://jsfiddle.net/RiOS/b6ned218/>



D3-style JavaScript

- ```
function widget(spec) {
 var instance = {};
```
- ```
  var description;  
  instance.description = function(d) {  
    if (!arguments.length) return description;  
    description = d;  
    return instance;  
  };
```
- ```
 instance.render = function() {
 var div = d3.select('body').append("div")
 .attr("class", "box");
 div.append("h3").text(headline)
 .attr("style", "color:" + spec.color);
 div.append("p").text(description);
 return instance;
 };
```
- ```
  return instance;  
}
```
- ```
var w = widget({ color: "brown" })
 .headline("Widget")
 .description("This is ... JavaScript.");
w.render();
```
- ```
<div class="box" style="color:brown">  
  <h3>Simple Widget</h3>  
  <p>This is ... JavaScript.</p>  
</div>
```
- <https://jsfiddle.net/RiOS/b6ned218/>

Internal private variable
of widget function object.



D3-style JavaScript

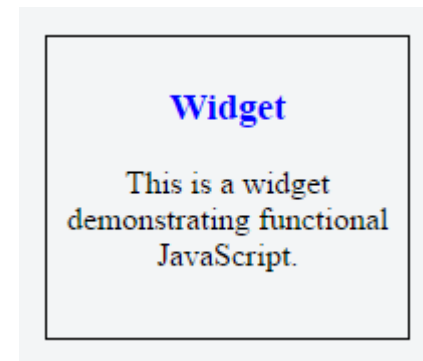
- ```
function widget(spec) {
 var instance = {};

 var description;

 instance.description = function(d) {
 if (!arguments.length) return description;
 description = d;
 return instance;
 };
```
- ```
instance.render = function() {  
  var div = d3.select('body').append("div")  
    .attr("class", "box");  
  div.append("h3").text(headline)  
    .attr("style", "color:" + spec.color);  
  div.append("p").text(description);  
  return instance;  
};
```
- ```
return instance;
}
```
- ```
var w = widget({ color: "brown" })  
  .headline("Widget")  
  .description("This is ... JavaScript.");  
w.render();
```
- ```
<div class="box" style="color:brown">
 Widget</h3>
 JavaScript.</p>
```

Private description variable is made available using getter-setter description function in instance.

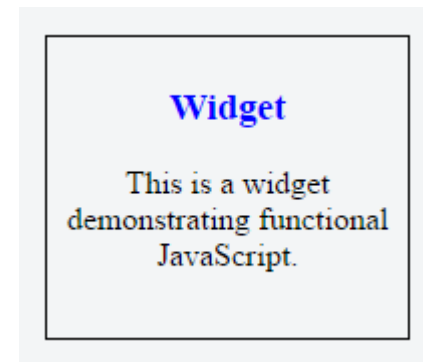
- <https://jsfiddle.net/RiOS/b6ned218/>



# D3-style JavaScript

- ```
function widget(spec) {  
  var instance = {};
```
- ```
 var description;
 instance.description = function(d) {
 if (!arguments.length) return description;
 description = d;
 return instance;
 };
```
- ```
  instance.render = function() {  
    var div = d3.select('body').append("div")  
      .attr("class", "box");  
    div.append("h3").text(headline)  
      .attr("style", "color:" + spec.color);  
    div.append("p").text(description);  
    return instance;  
  };
```
- ```
 return instance;
}
```
- ```
var w = widget({ color: "brown" })  
  .headline("Widget")  
  .description("This is ... JavaScript.");  
w.render();
```
- ```
<div class="box" style="color:brown">
 <h3>Simple Widget</h3>
 <p>This is ... JavaScript.</p>
</div>
```
- <https://jsfiddle.net/RiOS/b6ned218/>

Function also use method chaining design pattern.

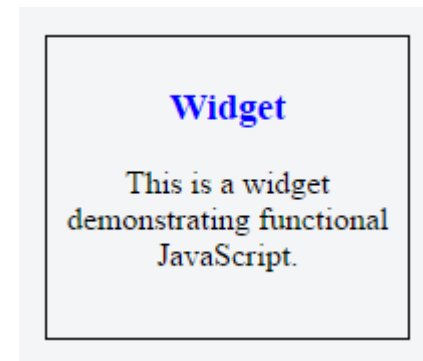




# D3-style JavaScript

- function widget(spec) {  
 var instance = {};
- var description;  
 instance.description = function(d) {  
 if (!arguments.length) return description;  
 description = d;  
 return instance;  
 };
- instance.render** = function() {  
 var div = d3.select('body').append("div")  
 .attr("style", "color: brown");  
 div.append("p").text(description);  
 **return instance;**  
};
- return instance;  
}
- var w = widget({ color: "brown" })  
 .headline("Widget")  
 .description("This is ... JavaScript.");  
w.render();
- <div class="box" style="color: brown">  
 <h3>Simple Widget</h3>  
 <p>This is ... JavaScript.</p>  
</div>
- <https://jsfiddle.net/RiOS/b6ned218/>

Public method with method chaining design pattern.



# Selections

- are a subclass of array
  - provides methods to manipulate selected elements
  - in fact they are arrays of arrays (more about that later)
- D3 uses CSS3 to select elements
  - e.g. you can select by
    - tag ("div")
    - class (".awesome")
    - unique identifier ("#foo")
    - attribute ("[color=red]")
    - containment ("parent child")
  - selectors can also be
    - intersected (".this.that" for logical AND)
    - unioned (".this, .that" for logical OR)
- if browser doesn't support selectors natively, include Sizzle before D3 for backwards-compatibility (<https://sizzlejs.com/>)

# What to do with selections?

- apply operators to them to do stuff, i.e. get or set
  - attributes
  - styles
  - properties
  - HTML and text content
- attribute values and such are specified as
  - constants
  - functions
    - evaluated for each element

# No loops to traverse selections

- to modify the document with D3
  - do not use for loops or recursive functions
  - use selections
    - they operate on entire selections
      - you do not have to do manual looping
    - you can still loop over individual elements
      - **each** operator invokes an arbitrary function on every element
      - selections are arrays
        - » elements can be accessed directly (e.g. `sel[0][0]`)
- D3 supports method chaining
  - for brevity when applying multiple operators
  - the operator return value is the selection

# Selecting elements

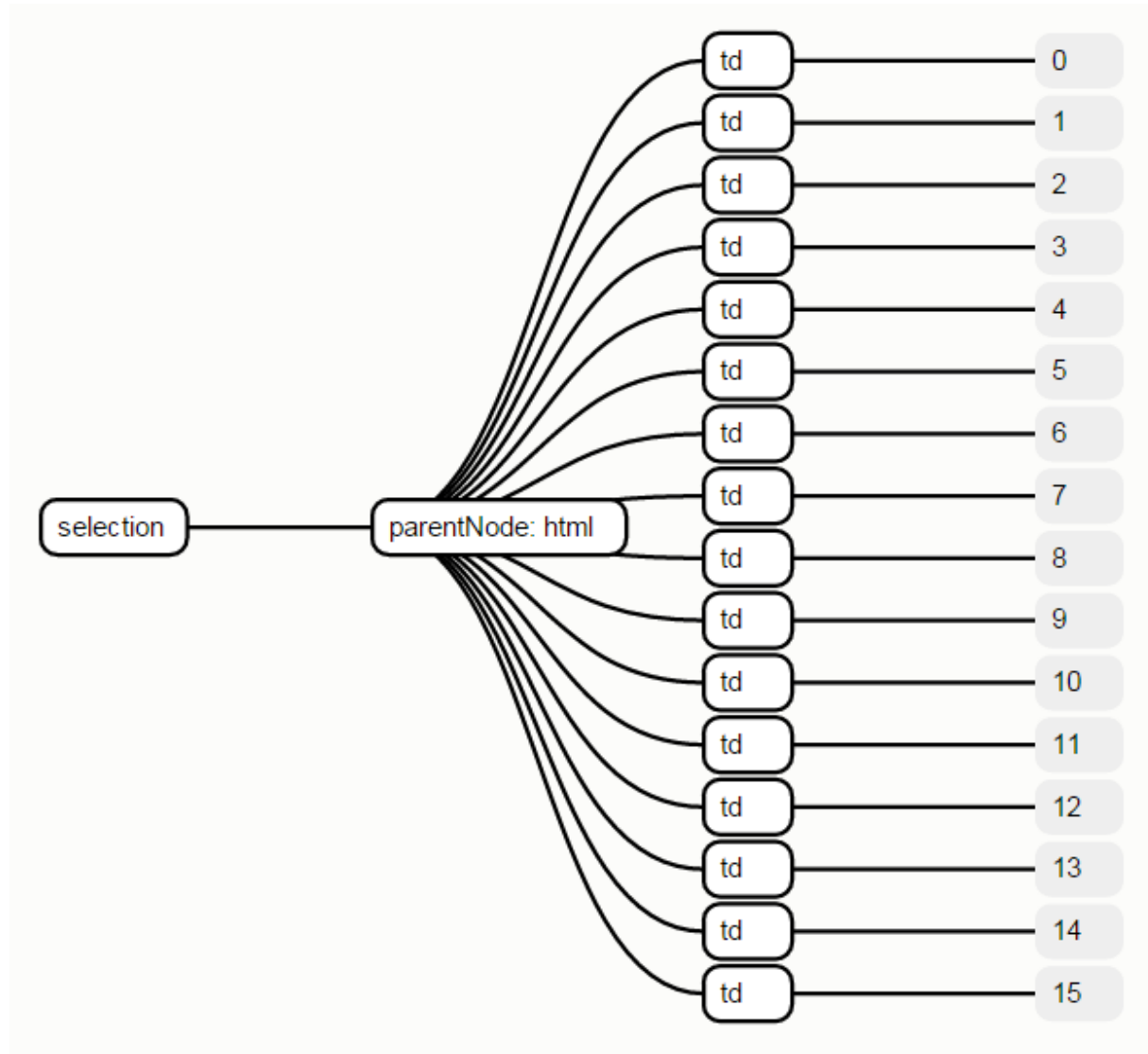
- Selecting single element:
  - HTML: `<p id="target"></p>`
  - JS: `d3.select("p#target").text("Hello world!");`
    - Hello world!
    - `d3.selectAll("p") ↦ Array [ Array [ <p> ] ]`
- Selecting multiple elements:
  - HTML: `<p></p><p></p>`
  - JS: `d3.selectAll("p").text("Hello world!");`
    - Hello world!
    - Hello world!
    - `d3.selectAll("p") ↦ Array [ Array [ <p>, <p> ] ]`

# Nested selections

- HTML:

```
<table>
 <thead>
 <tr><td> A</td><td> B</td><td> C</td><td> D</td></tr>
 </thead>
 <tbody>
 <tr><td> 0</td><td> 1</td><td> 2</td><td> 3</td></tr>
 <tr><td> 4</td><td> 5</td><td> 6</td><td> 7</td></tr>
 <tr><td> 8</td><td> 9</td><td>10</td><td>11</td></tr>
 <tr><td>12</td><td>13</td><td>14</td><td>15</td></tr>
 </tbody>
</table>
```
- `td1 = d3.selectAll("tbody td") ↦ Array [ Array[16] ]`
  - `td1[0].parentNode ↦ <html>`
  - `td2 = d3.select("tbody").selectAll("td") ↦ Array [ Array[16] ]`
  - `td2[0].parentNode ↦ <tbody>`

# d3.selectAll("tbody td")



# Nested selections

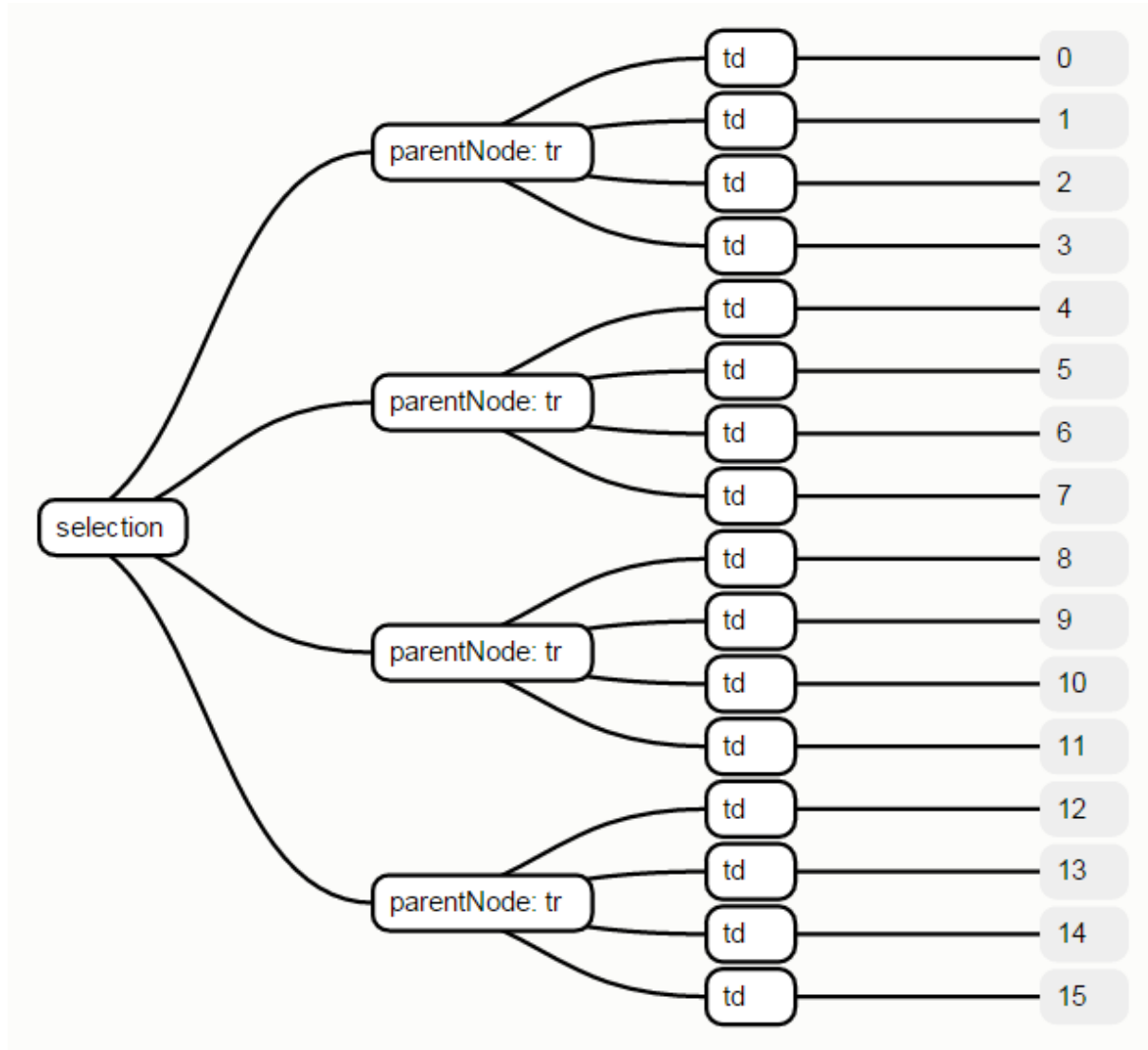
- HTML:

```
<table>
 <thead>
 <tr><td> A</td><td> B</td><td> C</td><td> D</td></tr>
 </thead>
 <tbody>
 <tr><td> 0</td><td> 1</td><td> 2</td><td> 3</td></tr>
 <tr><td> 4</td><td> 5</td><td> 6</td><td> 7</td></tr>
 <tr><td> 8</td><td> 9</td><td>10</td><td>11</td></tr>
 <tr><td>12</td><td>13</td><td>14</td><td>15</td></tr>
 </tbody>
</table>
```
- `td = d3.selectAll("tbody tr").selectAll("td")`  $\mapsto$   
Array [ Array[4], Array[4], Array[4], Array[4] ]
  - `td1[0].parentNode`  $\mapsto$  <tr>, ..., `td1[3].parentNode`  $\mapsto$  <tr>
- `td.style("color", function(d, i, j) {`  
  `return (i === 2) || (j === 1) ? "red" : null;`  
});

A	B	C	D
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15



`d3.selectAll("tbody tr").selectAll("td")`



# Joining data

- you can also join selections to data
  - this data is available to operators for data-driven transformations
- HTML:  
`<div id="divA"></div><div id="divB"></div>`
- `d3.selectAll("div")`  
    `.data(["prvy", "druhy"])`  
    `.text(function(d, i) { return d + i; });`
  - `divA.__data__`  $\mapsto$  "prvy"
  - `divB.__data__`  $\mapsto$  "druhy"
- in addition, joining to data produces
  - enter and exit sub-selections
- use them to add or remove elements
  - in response to changes in data

# General Update Pattern – no keys

- **HTML:**  

```
<p></p>
<div>Update: </div>
<div>Enter: </div>
<div>Exit: </div>
```
- ```
var i, words = ["Join", "new", "group.",
               "Say", "opacity", "now!"];
for (i = 0; i < words.length; i++)
  words[i] = words[i].split("");
```
- ```
function id(d) { return d; }
function i32(d, i) { return 2 + i * 32 + "px"; }
```
- ```
dura = 0; paus = 1000;
```
- ```
var p = d3.select("body").append("p");
```
- **// The initial display.**  

```
index = 0;
update(words[index]);
```
- **// Periodically show next word.**  

```
setInterval(function() {
 index++;
 if (index >= words.length) index = 0;
 update(words[index]);
}, dura + paus);
```
- ```
function update(data) {
  // DATA JOIN: Join new data with old elements, if any.
  var span = p.selectAll("span").data(data);

  d3.select("#update").text(span);
  d3.select("#enter").text(span.enter());
  d3.select("#exit").text(span.exit());

  // UPDATE: Update old elements as needed.
  span.attr("class", "update");

  // ENTER: Create new elements as needed.
  span.enter().append("span")
    .attr("class", "enter")
    .style("top", "34px")
    .style("left", i32);

  // ENTER + UPDATE: Appending to the enter selection
  // expands the update selection to include entering
  // elements. Operations on the update selection after
  // this will apply to both entering and updating nodes.
  span.text(id);

  // EXIT: Remove old elements as needed.
  span.exit().remove();
}
```

GUP – no keys

- ```
function update(data) {
 // DATA JOIN: Join new data with old elements, if any.
 var span = p.selectAll("span").data(data);

 d3.select("#update").text(span);
 d3.select("#enter").text(span.enter());
 d3.select("#exit").text(span.exit());

 // UPDATE: Update old elements as needed.
 span.attr("class", "update");

 // ENTER: Create new elements as needed.
 span.enter().append("span")
 .attr("class", "enter")
 .style("top", "34px")
 .style("left", i32);

 // ENTER + UPDATE
 // Appending to the enter selection
 span.text(id);

 // EXIT: Remove old elements as needed.
 span.exit().remove();
}
```

## GUP – with keys

- ```
function update(data) {  
  // DATA JOIN: Join new data with old elements, if any.  
  var span = p.selectAll("span").data(data, id);  
  
  d3.select("#update").text(span);  
  d3.select("#enter").text(span.enter());  
  d3.select("#exit").text(span.exit());  
  
  // UPDATE: Update old elements as needed.  
  span.attr("class", "update");  
  
  // ENTER: Create new elements as needed.  
  span.enter().append("span")  
    .attr("class", "enter")  
    .style("top", "34px")  
    .text(id);  
  
  // ENTER + UPDATE  
  // Appending to the enter selection  
  span.style("left", i32);  
  
  // EXIT: Remove old elements as needed.  
  span.exit().remove();  
}
```

GUP – with keys

- function update(data) {
 // DATA JOIN: Join new data with old elements, if any.
 var span = p.selectAll("span").data(data, id);
- d3.select("#update").text(span);
d3.select("#enter").text(span.enter());
d3.select("#exit").text(span.exit());
- // UPDATE: Update old elements as needed.**
span.attr("class", "update")
 .style("left", i32);

// ENTER: Create new elements as needed.
span.enter().append("span")
 .attr("class", "enter")
 .style("top", "34px")
 .style("left", i32)
 .text(id);

// EXIT: Remove old elements as needed.
span.exit()

 .remove();
}

GUP – with animation

- function update(data) {
 // DATA JOIN: Join new data with old elements, if any.
 var span = p.selectAll("span").data(data, id);
- d3.select("#update").text(span);
d3.select("#enter").text(span.enter());
d3.select("#exit").text(span.exit());
- // UPDATE: Update old elements as needed.**
span.attr("class", "update")
 .transition().ease("linear").duration(dura)
 .style("left", i32);

// ENTER: Create new elements as needed.
span.enter().append("span")
 .attr("class", "enter")
 .style("top", "2px")
 .style("left", i32)
 .style("opacity", 1e-6)
 .text(id)
 .transition().ease("linear").duration(dura)
 .style("top", "34px")
 .style("opacity", 1);

// EXIT: Remove old elements as needed.
span.exit().attr("class", "exit")
 .transition().ease("linear").duration(dura)
 .style("top", "66px")
 .style("opacity", 1e-6)
 .remove();
}

GUP – with animation and SVG

- classical HTML without SVG:
- ```
function update(data) {
 // DATA JOIN: Join new data with old elements, if any.
 var span = p.selectAll("span").data(data, id);

 // UPDATE: Update old elements as needed.
 span.attr("class", "update")
 .transition().ease("linear").duration(dura)
 .style("left", i32);

 // ENTER: Create new elements as needed.
 span.enter().append("span")
 .attr("class", "enter")
 .style("top", "2px")
 .style("left", i32)
 .style("opacity", 1e-6)
 .text(id)
 .transition().ease("linear").duration(dura)
 .style("top", "34px")
 .style("opacity", 1);

 // EXIT: Remove old elements as needed.
 span.exit().attr("class", "exit")
 .transition().ease("linear").duration(dura)
 .style("top", "66px")
 .style("opacity", 1e-6)
 .remove();
}
```
- ```
var width = 300, height = 100;  
var svg = d3.select("body").append("svg")  
  .attr("width", width).attr("height", height)  
  .append("g")  
  .attr("transform", "translate(16," + (height / 2) + ")");  
  
function update(data) {  
  // DATA JOIN: Join new data with old elements, if any.  
  var text = svg.selectAll("text").data(data, id);  
  
  // UPDATE: Update old elements as needed.  
  text.attr("class", "update")  
    .transition().ease("linear").duration(dura)  
    .attr("x", i32);  
  
  // ENTER: Create new elements as needed.  
  text.enter().append("text")  
    .attr("class", "enter")  
    .attr("dy", 12).attr("y", -80)  
    .attr("x", i32)  
    .text(id)  
    .transition().ease("linear").duration(dura)  
    .attr("y", 0);  
  
  // EXIT: Remove old elements as needed.  
  text.exit().attr("class", "exit")  
    .transition().ease("linear").duration(dura)  
    .attr("y", 80)  
    .remove();  
}
```

Interpolation pitfalls

- avoid interpolating to or from 0
 - if the interpolator is used to generate a string
 - such as with attr
 - very small values, when stringified, are converted to scientific notation
 - 0.0000001 is converted to the string "1e-7"
 - causes a temporarily invalid attribute or style property
- to avoid scientific notation
 - start or end the transition at 1e-6 (remember $\varepsilon = 1e-6$)
 - the smallest value that is not stringified in scientific notation

Nesting and Data

- HTML: the same table
- ```
var matrix = [
 [1, 12, 3, 0],
 [4, 11, 6, 7],
 [10, 5, 8, 9],
 [15, 2, 13, 14],
];
```
- ```
function id(d) { return d; }
```
- ```
var td = d3.selectAll("tbody tr")
 .data(matrix)
 .selectAll("td")
 .data(id) // d <- matrix[row]
 .text(id);
```

A	B	C	D
1	12	3	0
4	11	6	7
10	5	8	9
15	2	13	14



# Create table from scratch

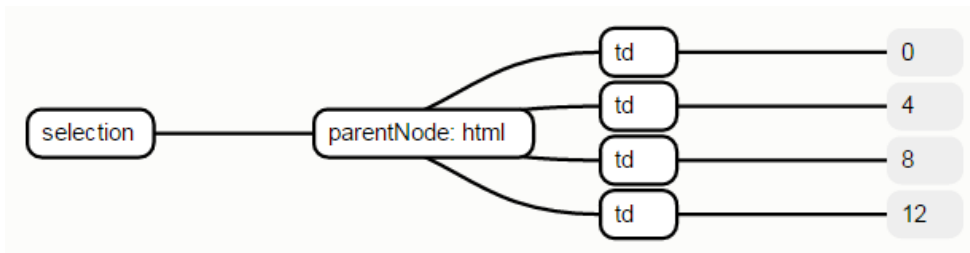
- ```
var matrix = [  
  [ 1, 12, 3, 0],  
  [ 4, 11, 6, 7],  
  [10, 5, 8, 9],  
  [15, 2, 13, 14],  
];
```
- ```
function id(d) { return d; }
```
- ```
var body = d3.select("body");  
var table = body.append("table");  
var tr = table.selectAll("tr")  
  .data(matrix)  
  .enter().append("tr");
```
- ```
var td = tr.selectAll("td")
 .data(id)
 .enter().append("td")
 .text(id);
```

1	12	3	0
4	11	6	7
10	5	8	9
15	2	13	14

# Nesting: select vs. selectAll

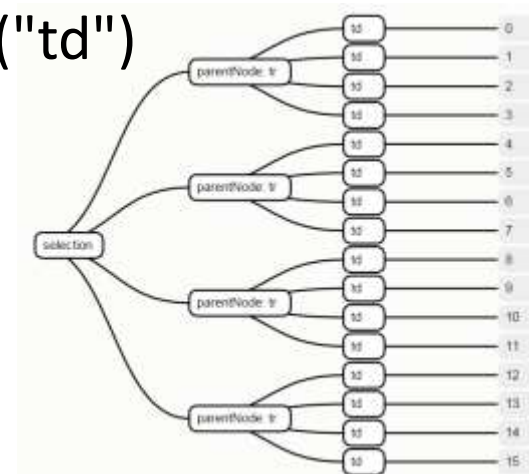
## select

- preserves the existing grouping
  - the data
  - Index
  - even the parent node of the original selection
- `d3.selectAll("tbody tr").select("td")`



## selectAll

- creates a new grouping
- the only way to obtain a nested selection
  - call `selectAll` on an existing selection
- `d3.selectAll("tbody tr").selectAll("td")`



# Scales

- scales are mapping functions (input domain  $\mapsto$  output range)
  - the input domain
    - typically a visualized dimension of the data
      - e.g. the height of students (measured in meters)
  - the output range
    - typically a dimension of the desired output visualization
      - e.g. the height of bars (measured in pixels)
- quantitative scales have a continuous domain
  - e.g. the set of real numbers or dates
- ordinal scales have a discrete domain
  - e.g. a set of names or categories
- scales are optional you don't have to use them
  - if you prefer to do the math yourself
  - however, using scales simplify the code needed to map data to a visual representation

# Linear Scales

- the most common scale
- a good default choice to map a continuous input domain to a continuous output range
  - the mapping is linear:  $y = a x + b$
- `var x = d3.scale.linear()`
  - constructs a new linear scale with the default domain `[0,1]` and the default range `[0,1]`
- `x.domain([min,max])` – min, max convertible to numbers
- `x.range([a,b])` – a, b interpolable by output interpolator
- polylinear scale:
  - `var color = d3.scale.linear().domain([-1, 0, 1])`  
`color.range(["red", "white", "green"]);`
  - `color(0.5) ↦ #80c080`

# SVG shapes

- a number of built-in simple shapes
  - e.g. axis-aligned rectangles and circles
- all shapes can be
  - transformed (e.g. rotated, moved)
    - using the transform attribute
  - filled and stroked
    - using the fill and stroke styles
- `svg:rect x="0" y="0" width="0" height="0" rx="0" ry="0"`
- `svg:circle cx="0" cy="0" r="0"`
- `svg:line x1="0" y1="0" x2="0" y2="0"`
- `svg:text x="0" y="0" dx="0" dy="0" text-anchor="start"`
- `svg:path d=""` – generalization of all other shapes in SVG

# Path Data Generators

- helper classes for generating path data
- e.g. define a line generator:

```
var line = d3.svg.line()
 .x(function(d) { return d.x; })
 .y(function(d) { return d.y; })
 .interpolate("cardinal");
```
- Use line to set the d attribute:

```
g.append("path").attr("d", line);
```

  - `<path d="M0,124Q12,81.4,15,72C19.5,57.9,25.5,40.2,30,30S40.35,8.5,45,4S56.35,-1.95,61,0S71.5,9.05,76,17S86.5,40.70,91,53S101.5,84.75,106,99S ... Q443,126,455,124"></path>`
  - use data bound to g or explicitly specify with `line(data)`

# SVG line

- `var line = d3.svg.line()`
  - constructs a new line generator
    - the default x- and y-accessor functions
      - assume the input data is a two-element array of numbers
    - linear interpolation
- `line(data)`
  - returns the path data string for the specified array
- `line.x(accessor), line.y(accessor)`
  - sets the accessor to the specified function
- `line.interpolate(interpolate_mode)`
  - linear, cardinal, ...

# D3's axis component

- displays reference lines for scales automatically
  - takes care of the tedious task of drawing axes and labeled ticks
- `var axis = d3.svg.axis()` – create a new default axis
- `axis.scale(scale)` – sets the scale for axis
  - uses domain and range to draw and populate axis
- `axis.orient(orientation)`
  - horizontal axis
    - "top" – ticks above the domain path
    - "bottom" – ticks below the domain path
  - vertical axis
    - "left" – ticks to the left of the domain path
    - "right" – ticks to the right of the domain path
- `svg.append("g").call(axis)`
  - insert the axis into a selection
    - must contain an svg or g element



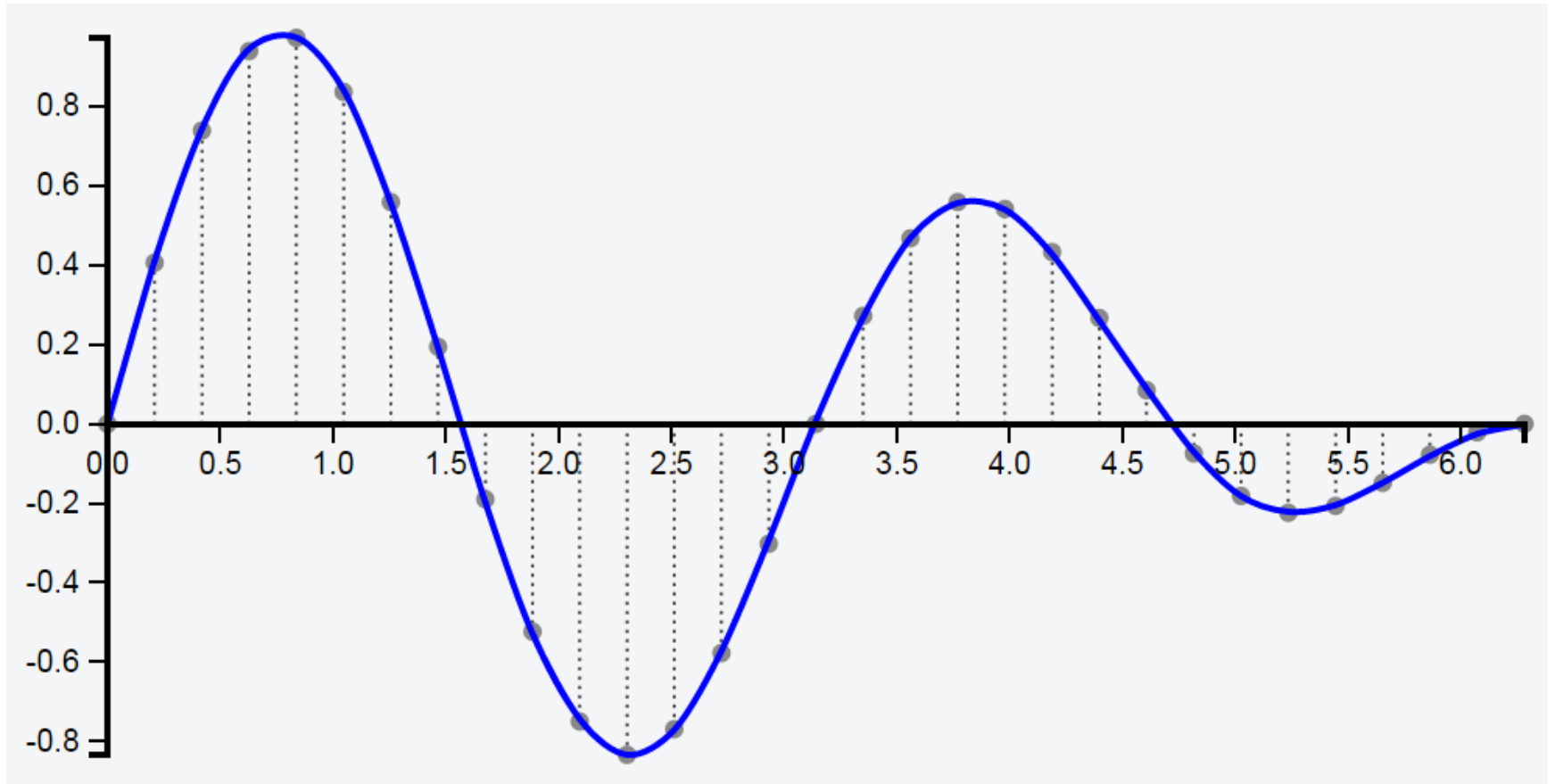
# SVG graph – preparation

- ```
function f(x) {  
  return Math.sin(2 * x) * Math.cos(x / 4);  
}
```
- ```
var steps = 30, delta = 2 * Math.PI / steps;
var data = [];
for (var i = 0, t = 0; i <= steps; i++, t += delta)
 data.push({ x: t, y: f(t) });
```
- ```
function getX(d) { return d.x; }  
function getY(d) { return d.y; }
```
- ```
var margin = {
 top: 10, right: 10, bottom: 10, left: 35
};
var width = 500, height = 250;
```
- ```
var  
  inWidth = width - margin.left - margin.right,  
  inHeight = height - margin.top - margin.bottom;
```
- ```
var svg = d3.select("body")
 .append("svg")
 .attr("width", width)
 .attr("height", height)
 .append("g")
 .attr("transform", "translate(" + margin.left +
 ", " + margin.top + ")");
```
- ```
var x = d3.scale.linear()  
  .domain([d3.min(data, getX),  
    d3.max(data, getX)])  
  .range([0, inWidth]);
```
- ```
var y = d3.scale.linear()
 .domain([d3.min(data, getY),
 d3.max(data, getY)])
 .range([inHeight, 0]);
```
- ```
var line = d3.svg.line()  
  .x(function(d) { return x(d.x); })  
  .y(function(d) { return y(d.y); })  
  .interpolate("cardinal");
```

SVG graph – main part

- var bars = svg.selectAll(".bar")
 .data(data, getX);
- var bar = bars.enter()
 .append("g")
 .attr("class", "bar")
 .attr("transform", function(d) {
 return "translate(" + x(d.x) + ",0)";
 });
- bar.append("line")
 .attr("y1", y(0))
 .attr("y2", function(d) { return y(d.y); });
- bar.append("circle")
 .attr("cy", function(d) { return y(d.y); });
- svg.append("path")
 .attr("class", "line")
 .attr("d", line(data));
- var xAxis = d3.svg.axis()
 .scale(x)
 .orient("bottom");
- svg.append("g")
 .attr("class", "x axis")
 .attr("transform",
 "translate(0," + y(0) + ")")
 .call(xAxis);
- var yAxis = d3.svg.axis()
 .scale(y)
 .orient("left");
- svg.append("g")
 .attr("class", "y axis")
 .call(yAxis);

SVG graph



Slovak locale

- ```
var sk_SK = {
 "decimal": ",", // decimal place string
 "thousands": "\u202F", // group separator string (narrower non-breaking space)
 "grouping": [3], // array of group sizes, cycled as needed
 "currency": ["", "\u202F€"], // currency prefix and suffix strings
 "dateTime": "%a, %e %b %Y, %X",
 "date": "%d.%m.%Y",
 "time": "%H:%M:%S",
 "periods": ["AM", "PM"], // unused
 "days": ["Nedeľa", "Pondelok", "Utorok", "Streda", "Štvrtok", "Piatok", "Sobota"],
 "shortDays": ["Ne", "Po", "Ut", "St", "Št", "Pi", "So"],
 "months": ["Január", "Február", "Marec", "Apríl", "Máj", "Jún",
 "Júl", "August", "September", "Október", "November", "December"],
 "shortMonths": ["Jan", "Feb", "Mar", "Apr", "Máj", "Jún",
 "Júl", "Aug", "Sep", "Okt", "Nov", "Dec"]
};
```
- ```
var locale = d3.locale(sk_SK);
```
- ```
locale.numberFormat("specifier") // locale's equivalent of d3.format
```
- ```
locale.timeFormat("specifier") // locale's equivalent of d3.time.format
```

Number formatting

- format specifier is modeled after Python 3.1's built-in [format specification mini-language](#)
 - `[[fill]align][sign][symbol][0][width][,][.precision][type]`
- `var nf = locale.numberFormat("+$, .2f");`
 - `sign = +` sign even for positive numbers
 - `symbol = $` use currency symbol (per the locale)
 - `,` signals the use of a thousands separator
 - `.2f` fixed-point number, 2 digits after the decimal point
- `nf(1234567.235) ↦ "+1 234 567,24 €"`

Time formatting

- `var tf = locale.timeFormat("%e %b %Y, %X");`
- `tf(new Date(2015, 4, 1, 2, 3, 4)) ↦`
`" 1 Máj 2015, 02:03:04"`
- `tf.parse("1 Máj 2015, 02:03:04").getDate()`
 - returns 1
 - `getDate()` returns the day of the month (from 1-31)
 - `getDay()` returns the day of the week (from 0 sun. - 6)
 - `getMonth()` returns the month (from 0-11)
- `tf = locale.timeFormat("%H:%M:%S");`
- `tf(new Date()) ↦ "22:09:10"`